

TENSOR AND SEQUENCE PARALLELISM SLIDES

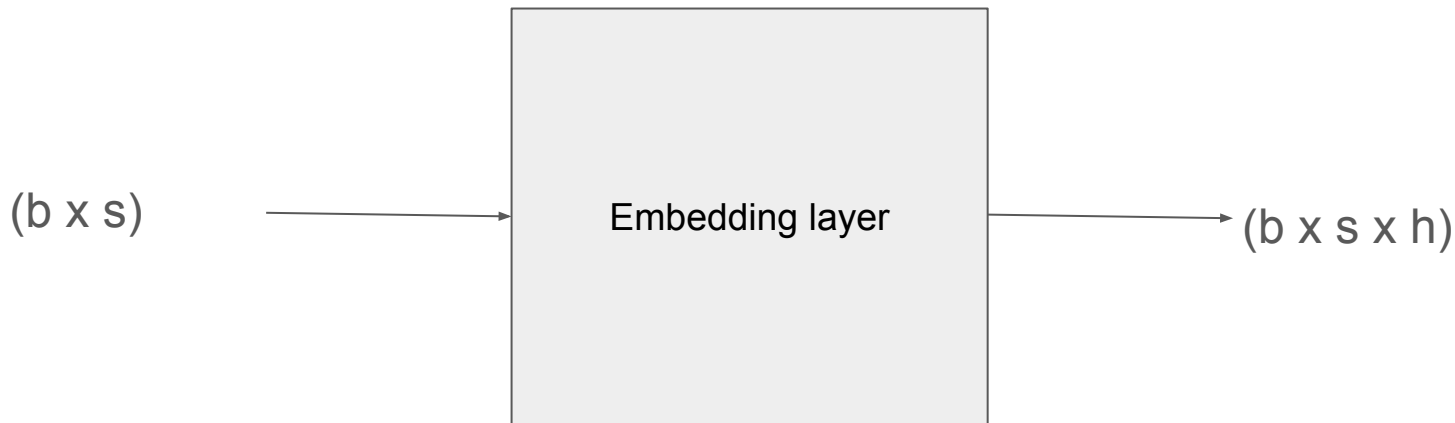
Mini transformer and total activations memory

Notations

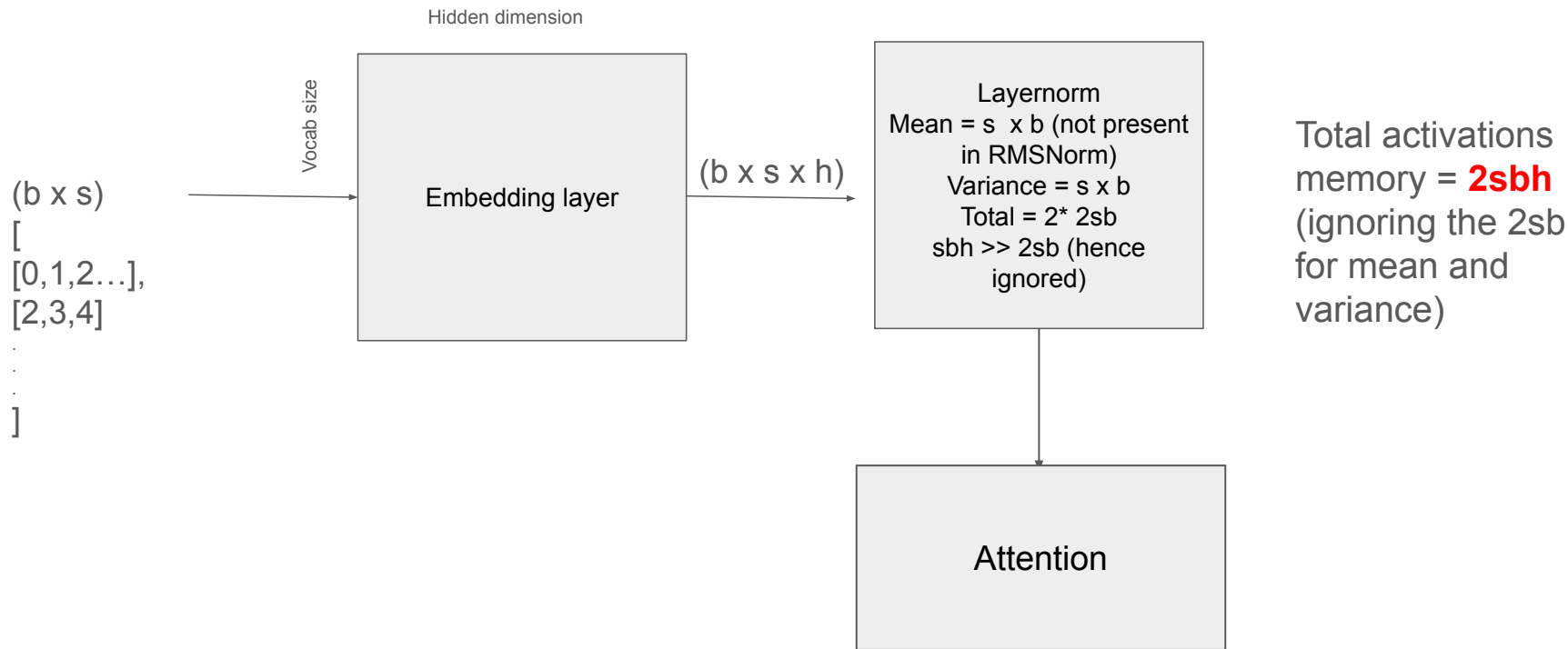
- a = number of attention heads
- b = microbatch size
- h = hidden dimension size
- L = number of layers in transformer
- v = vocabulary size

Embedding layer

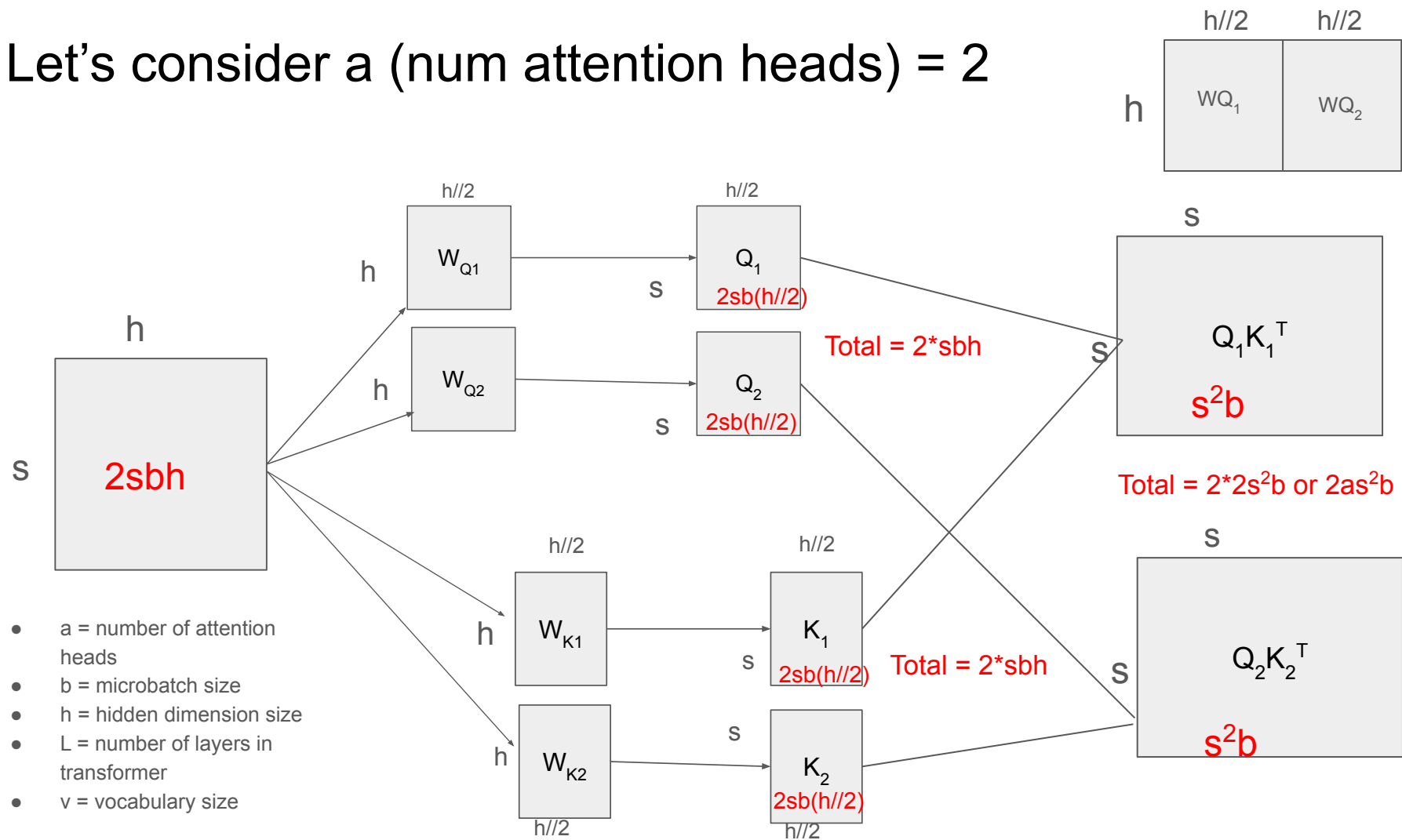
- a = number of attention heads
- b = microbatch size
- h = hidden dimension size
- L = number of layers in transformer
- v = vocabulary size



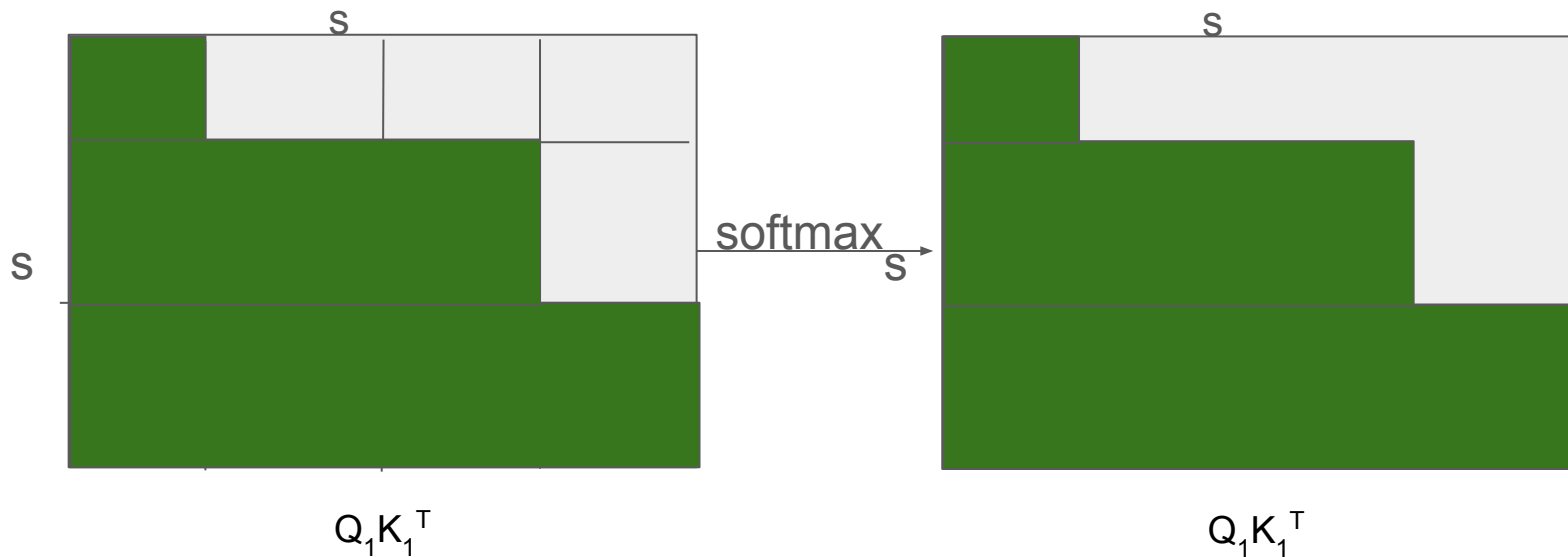
Layernorm and RMSNorm

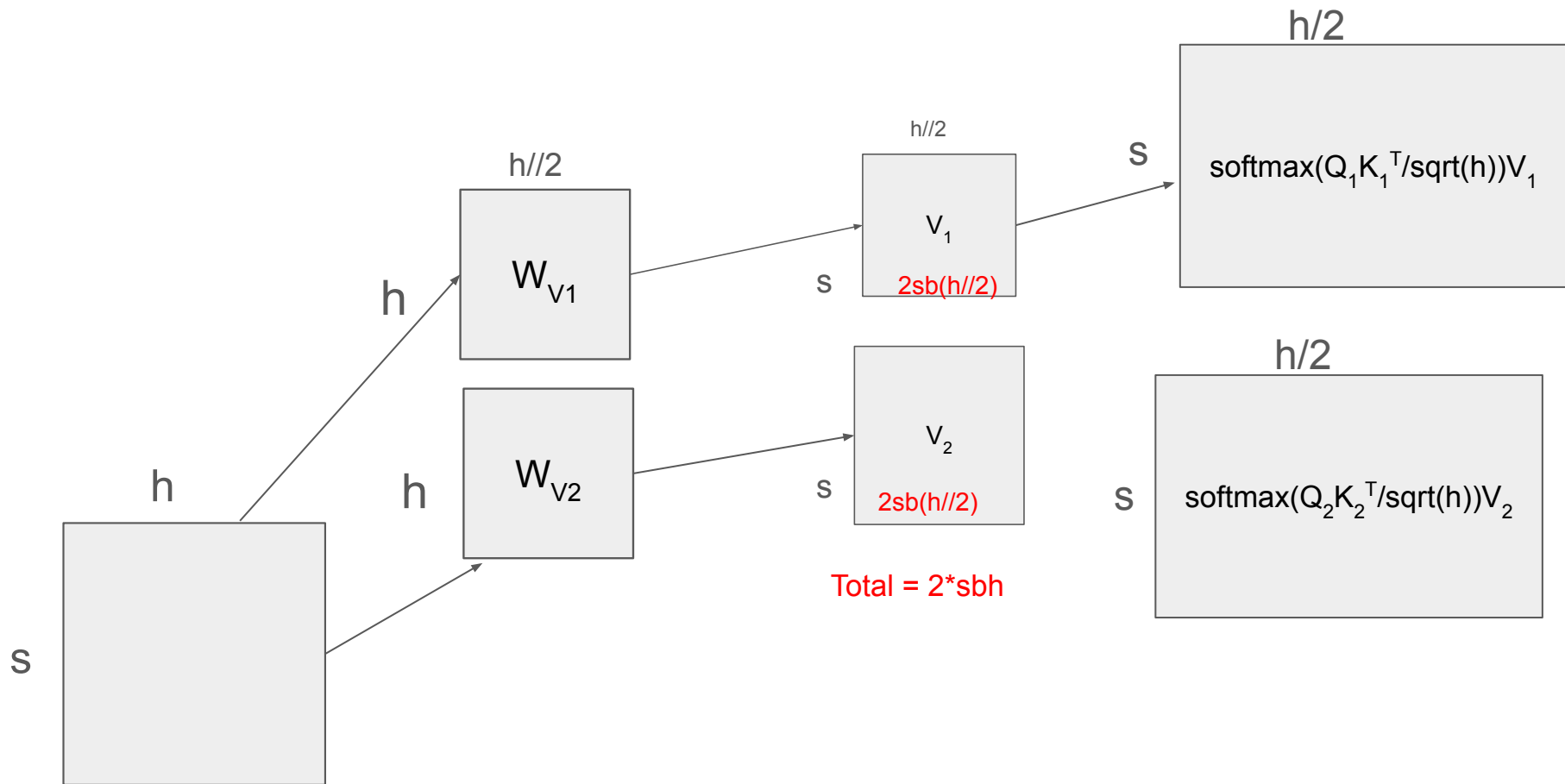


Let's consider a (num attention heads) = 2



Causal language model masking





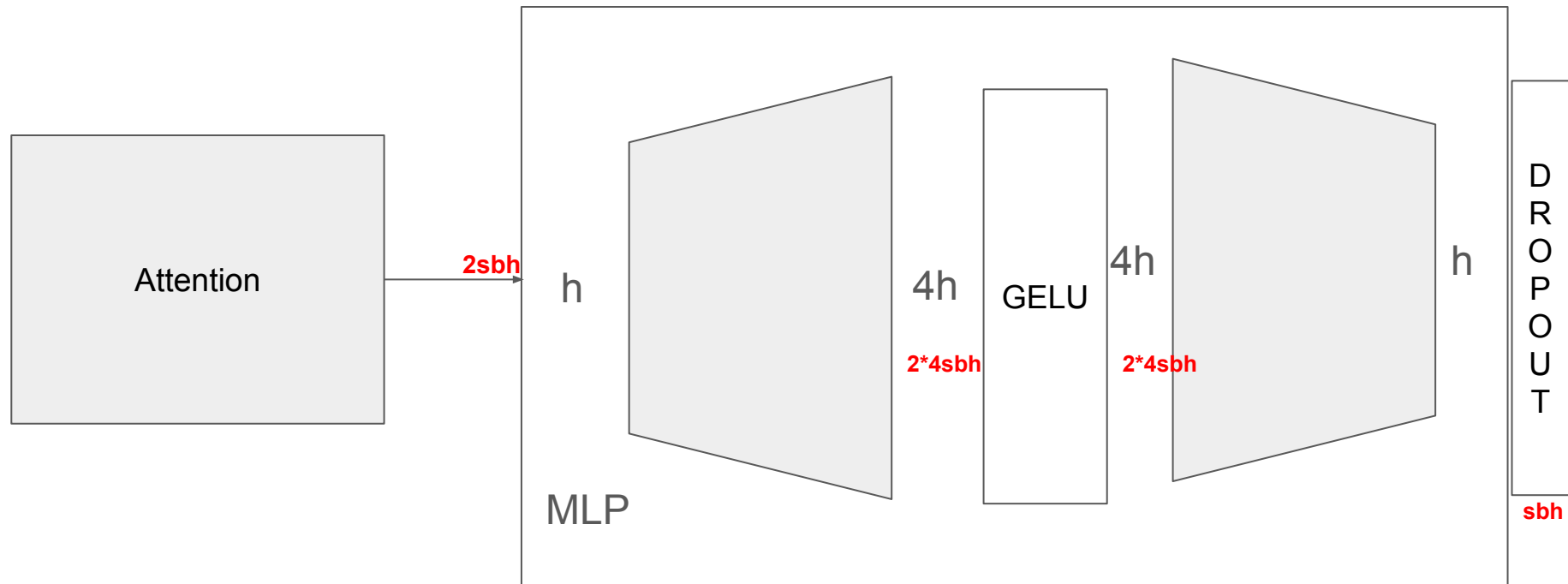
4.1 Activations Memory Per Transformer Layer

As shown in Figure 2, each transformer layer consists of an attention and an MLP block connected with two layer-norms. Below, we derive the memory required to store activations for each of these elements:

Attention block: which includes self attention followed by a linear projection and an attention dropout. The linear projection stores its input activations with size $2sbh$ and the attention dropout requires a mask with size sbh . The self attention shown in Figure 3 consists of several elements:

- **Query (Q), Key (K), and Value (V) matrix multiplies:** We only need to store their shared input with size $2sbh$.
- **QK^T matrix multiply:** It requires storage of both Q and K with total size $4sbh$.
- **Softmax:** Softmax output with size $2as^2b$ is required for back-propagation.
- **Softmax dropout:** Only a mask with size as^2b is needed.
- **Attention over Values (V):** We need to store the dropout output ($2as^2b$) and the Values ($2sbh$) and therefore need $2as^2b + 2sbh$ of storage.

MLP



TOTAL = 19sbh

MLP

$2sbh$

LAYERNORM
OR
RMSNORM

Mean = sb (not for
RMSNorm)

Variance = sb

It is dominated by sbh , hence ignored

Total activations

- Embeddings (**sbh**) -> Layernorm input: **2sbh**
- Dropout = **sbh**
- Attention
 - Shared inputs = **2sbh**
 - $Q = \mathbf{2sbh}$
 - $K = \mathbf{2sbh}$
 - $V = \mathbf{2sbh}$
 - $\text{Softmax}(QK^T) = 2as^2b$
- Dropout: **sbh**
- MLP + dropout
 - **2sbh**(input) + **8sbh**(up projection) + **8sbh** (gelu) + **sbh** (dropout) = **19sbh**
- Final layernorm input = **2sbh**

$$\text{Total} = 34sbh + 2as^2b$$

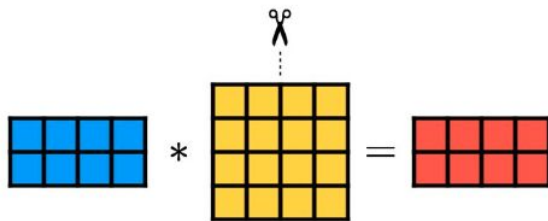
Reducing activations memory paper ([source](#))

- In this paper, the total activations memory is

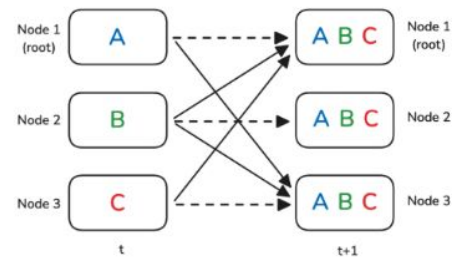
$$\text{Activations memory per layer} = sbh \left(34 + 5 \frac{as}{h} \right).$$

- The extra $3as^2b$ is from dropout that we ignored

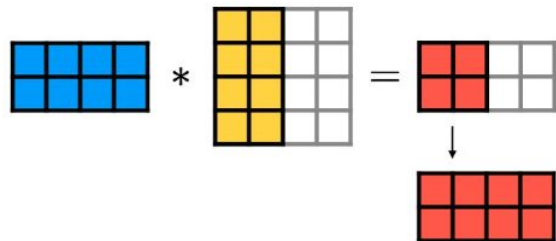
Tensor Parallelism (MLP) (Column parallel)



AllGather

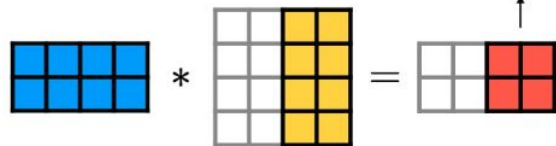


GPU 1

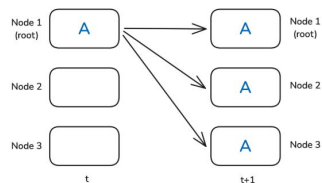


All-gather

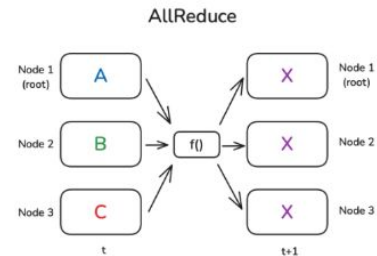
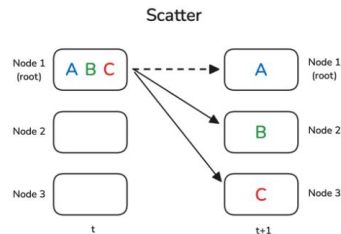
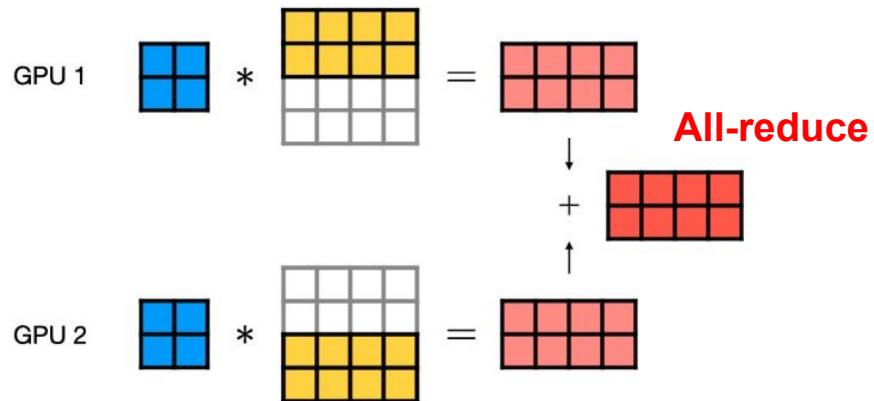
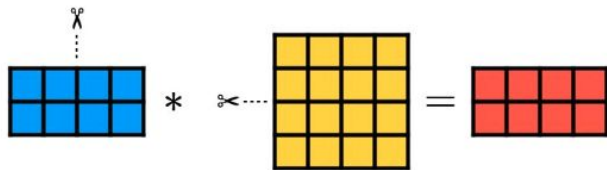
GPU 2



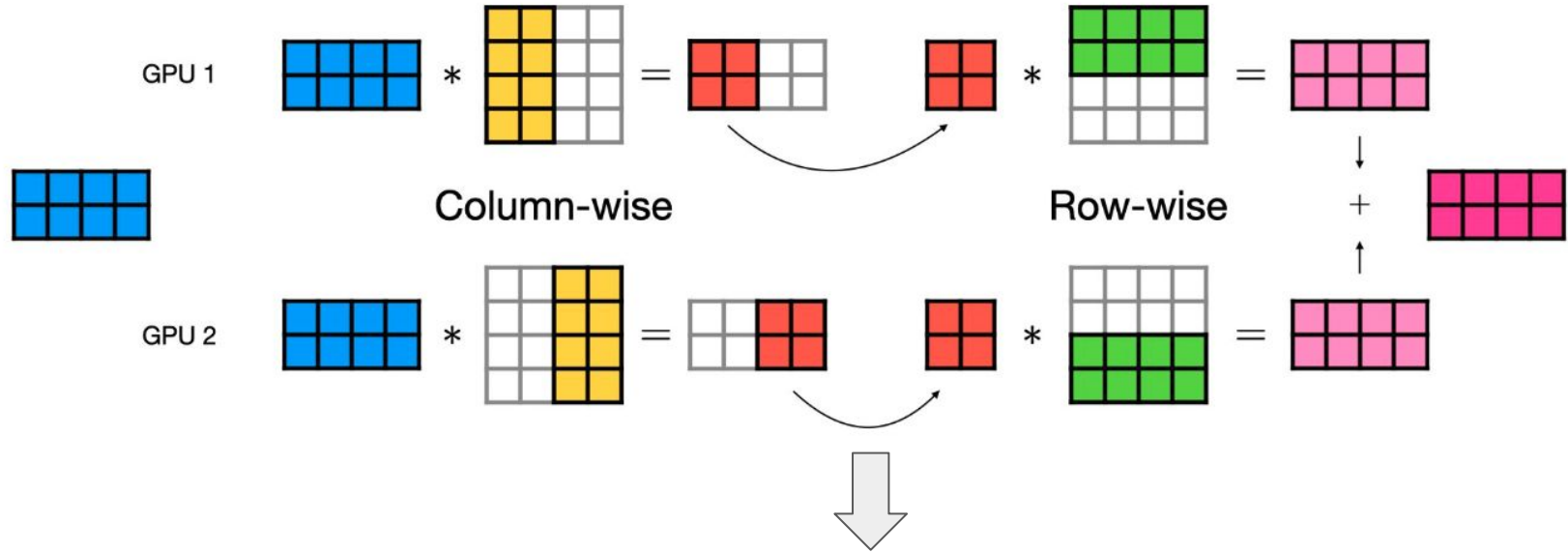
Broadcast



Row parallel



Column-Parallel -> Row-parallel



Apply GeLU activation function

4 combinations possible

1. Column $\rightarrow$ Row
2. Row $\rightarrow$ Column
3. Column $\rightarrow$ Column
4. Row $\rightarrow$ Row

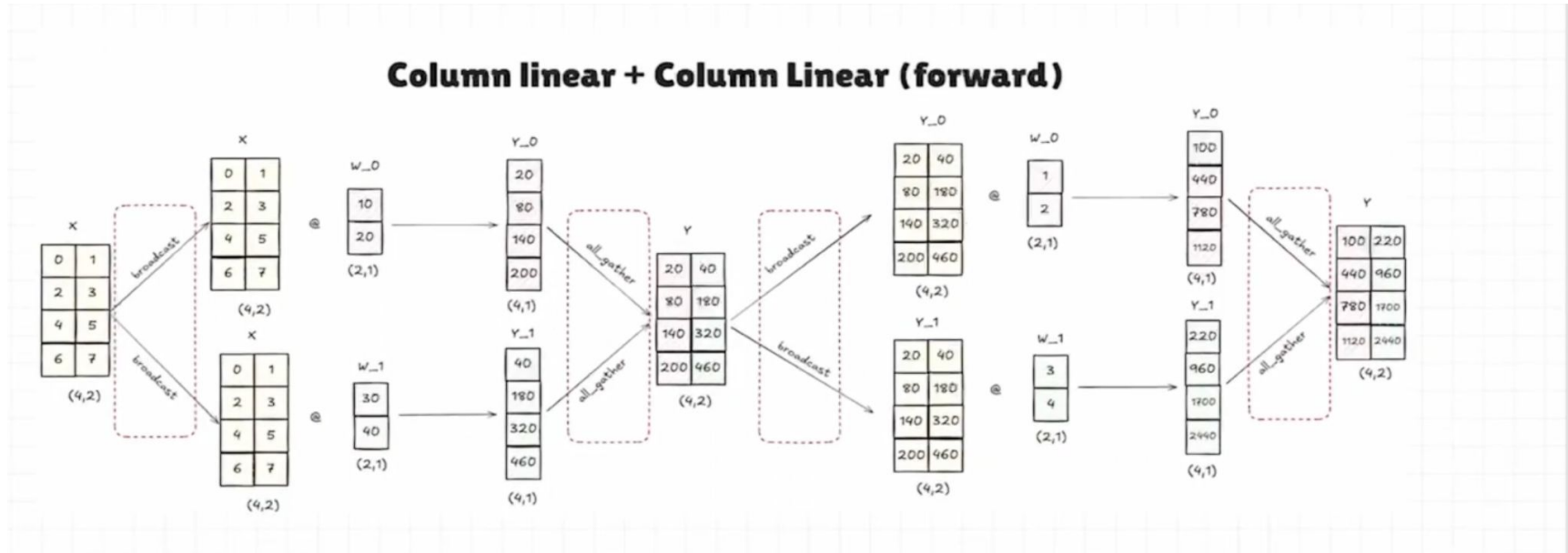
Identity mapping

- $\text{cat}(\text{GeLU}(XW_0), \text{GeLU}(XW_1)) = \text{GeLU}(\text{cat}(XW_0, XW_1))$
- $\text{GeLU}(XW_0) + \text{GeLU}(XW_1) \neq \text{GeLU}(XW_0 + XW_1)$

Hence, if we start with Row parallel, we need to sum (all-reduce) which will break the identity above. Hence we discard Row -> Col and Row -> Row

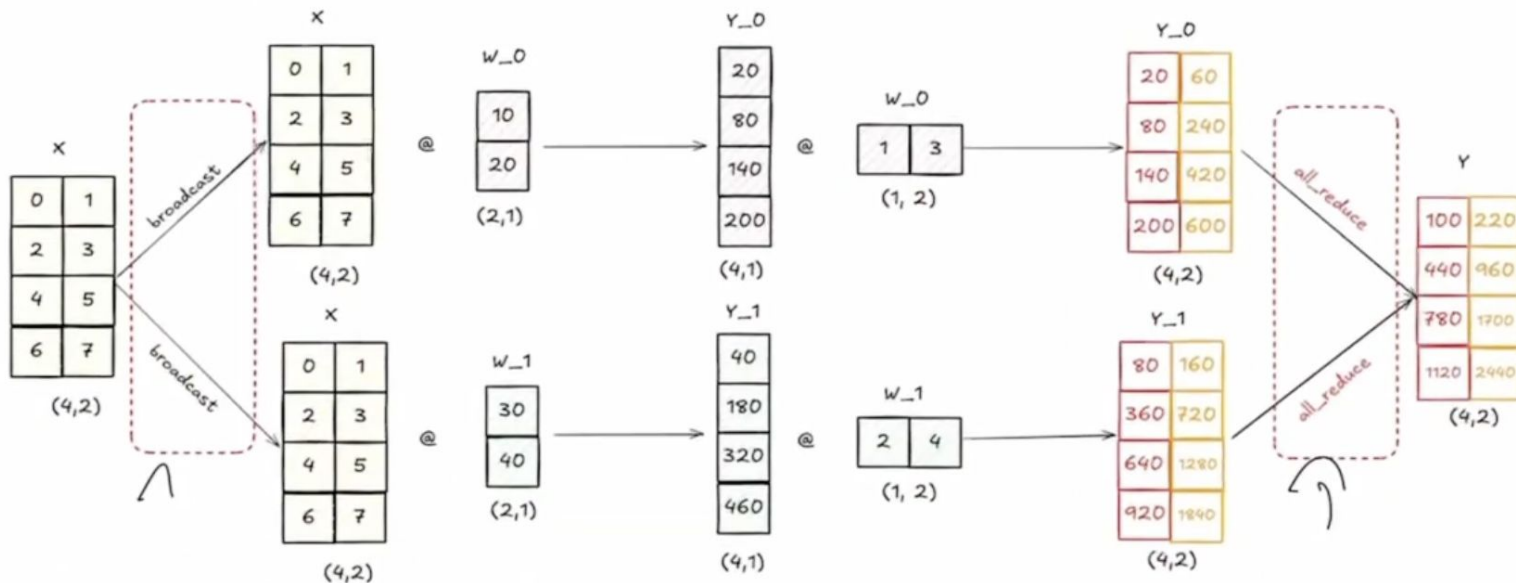
Hence only Col -> Row and Col -> Col remain

Column -> Column (4 communications 2 broadcast and 2 all gather)

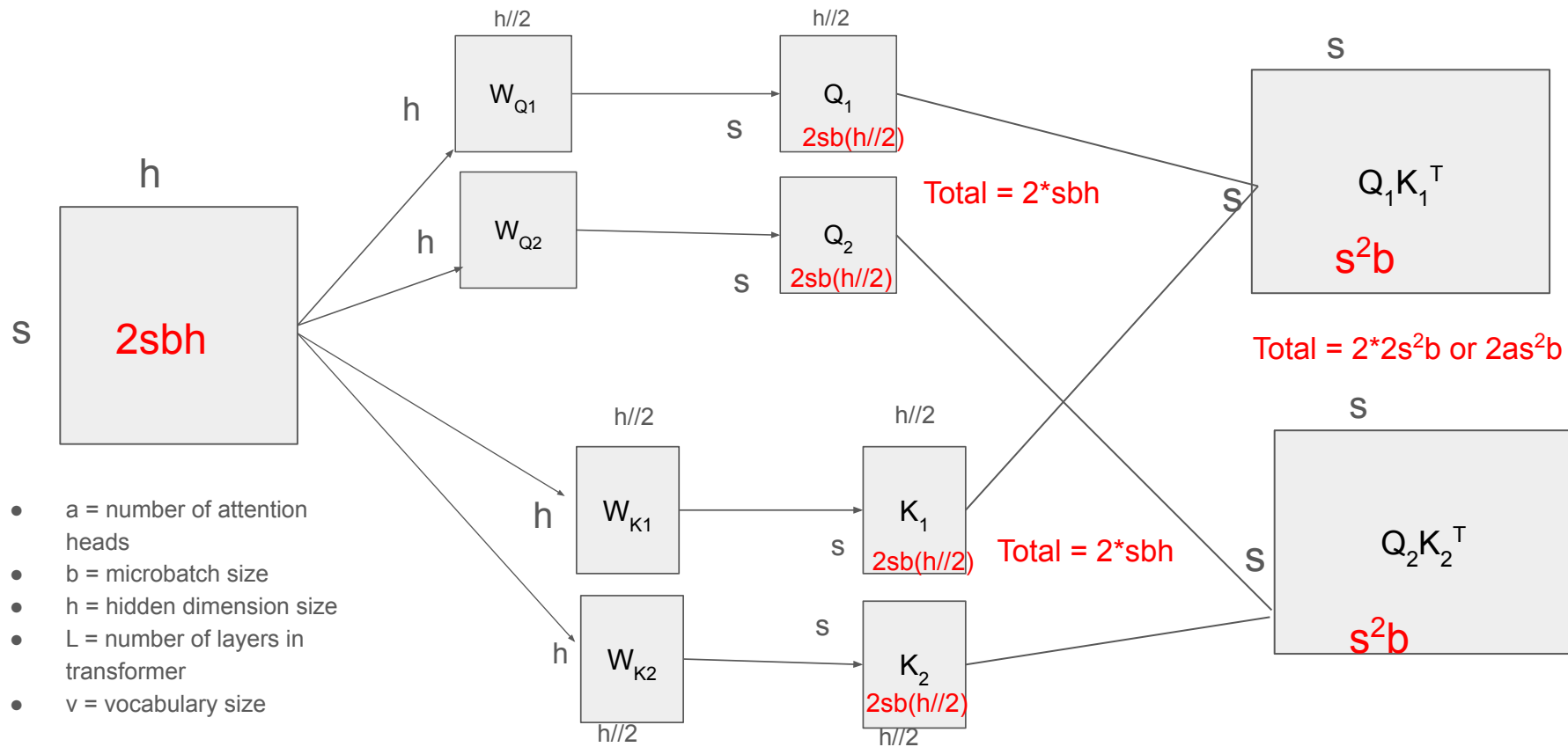


Column $\rightarrow$ Row (2 communications 1 broadcast and 1 all-reduce)

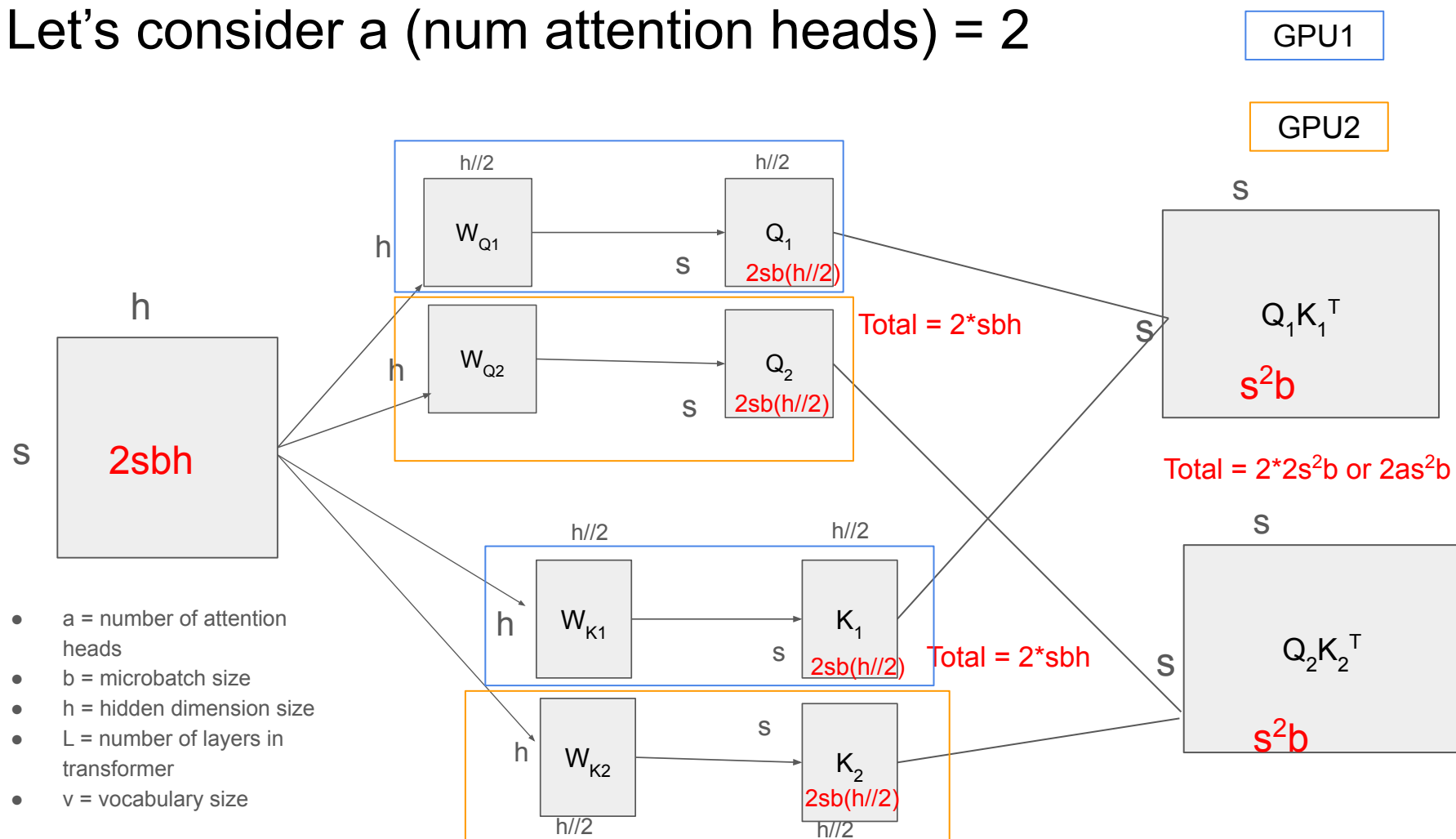
Column linear + Row Linear (forward)

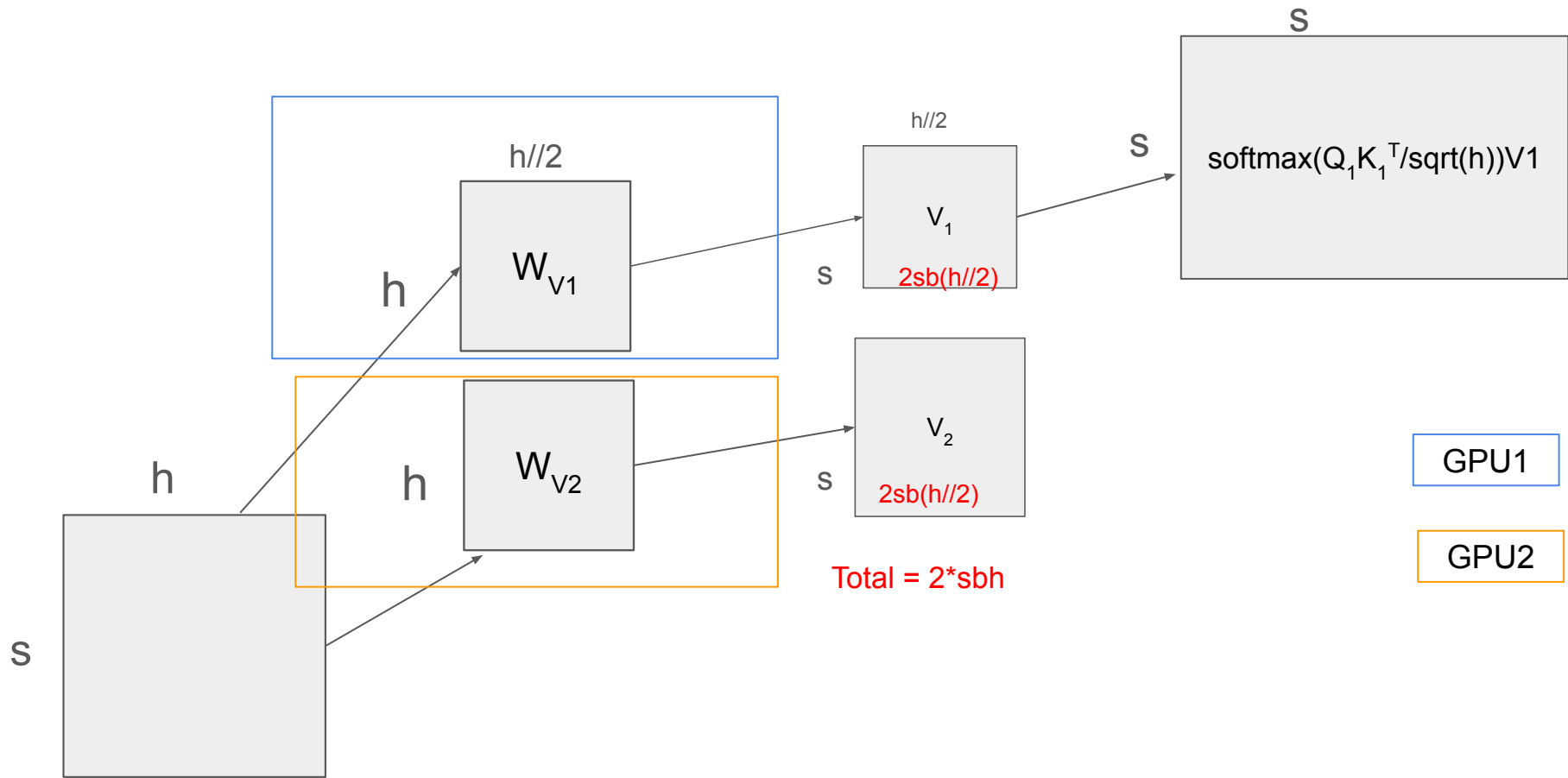


Let's consider a (num attention heads) = 2



Let's consider a (num attention heads) = 2





Vocab Parallel Embedding ([Source](#))

Embeddings = [10, 20, 30, 40, 50, 60] (vocab size = 6)

Index = [0, 1, 2, 3, 4, 5]

X = [1,4,2,5] (indices)

Emb(X) = [20, 50, 30, 60]

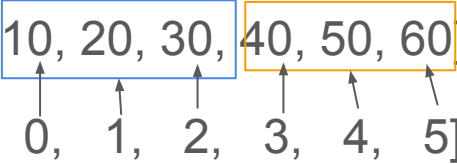
2 GPUs

GPU1

GPU2

Embeddings = [10, 20, 30, 40, 50, 60] (vocab size = 6)

Index = [0, 1, 2, 3, 4, 5]



$X = [1, 4, 2, 5]$ (indices)

$\text{Emb}(X) = [20, 50, 30, 60]$

$X = [1, 4, 2, 5]$ (indices)

Embeddings (GPU 1) = [10, 20, 30] (range 0 - 2)

- $X_GPU1 = X_mask * X = [True, False, True, False] * [1, 4, 2, 5] = [1, 0, 2, 0]$
- Subtract by lower range = $[max(1-0), max(0-0), max(2-0), max(0-0)]$
- $X[X_GPU1] = [20, 10, 30, 10]$
- $X[X_GPU1] * X_mask = [20, 0, 30, 0]$

Embeddings (GPU2) = [40, 50, 60] (range 3 - 5)

$X = [1, 4, 2, 5]$ (indices)

Embeddings (GPU 1) = $[10, 20, 30]$ (range 0 - 2)

- $X_GPU1 = X_mask * X = [True, False, True, False] * [1, 4, 2, 5] = [1, 0, 2, 0]$
- Subtract by lower range = $[max(1-0), max(0-0), max(2-0), max(0-0)]$
- $X[X_GPU1] = [20, 10, 30, 10]$
- **$X[X_GPU1] * X_mask = [20, 0, 30, 0]$**

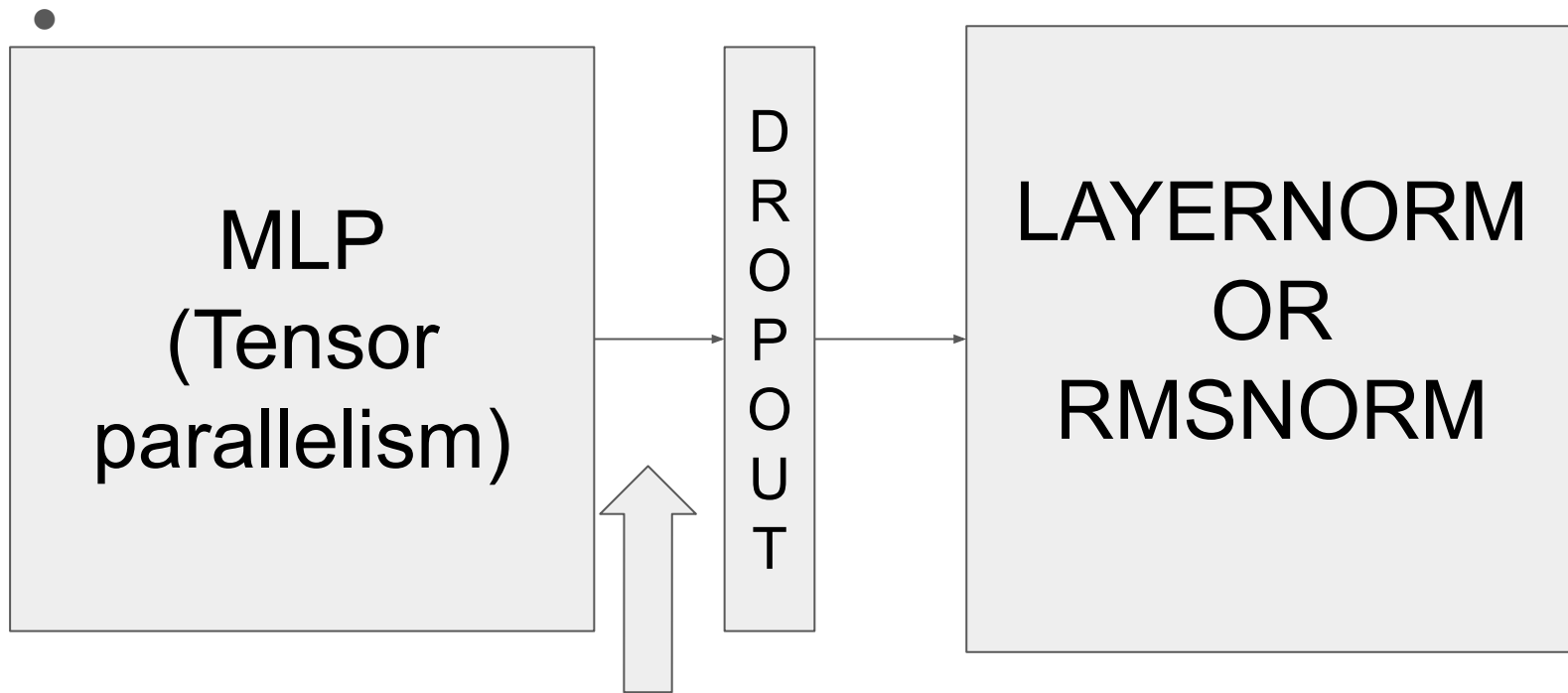
Embeddings (GPU2) = $[40, 50, 60]$ (range 3 - 5)

- $X_GPU2 = X_mask * X = [False, True, False, True] * [1, 4, 2, 5] = [0, 4, 0, 5]$
- Subtract by lower range = $[max(0-3), max(4-3), max(0-3), max(5-3)] = [0, 1, 0, 2]$
- $X[X_GPU2] = [40, 50, 40, 60]$
- **$X[X_GPU2] * X_mask = [0, 50, 0, 60]$**

All-reduce **$X[X_GPU1] * X_mask = [20, 0, 30, 0]$** and **$X[X_GPU2] * X_mask = [0, 50, 0, 60]$**

Final = $[20, 0, 30, 0] + [0, 50, 0, 60] = [20, 50, 30, 60]$

SEQUENCE PARALLELISM (Why) → **reduce peak activations memory**



This requires an expensive all-reduce across the sequence dimension

DROPOUT, LAYERNORM AND RESIDUAL BLOCK

- These two operations are performed independently on each token in a sequence
- Dimension = [b,s, h]
 - s = sequence length
 - b = micro batch size
 - h = hidden dimension
- Dropout
 - With seed, we generate the mask for the hidden dimension, and it is independent for each token
- Layernorm: Also independent for each token

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Source

